GRADUATION PROJECT · TECHNICAL DOCUMENTATION

Backend Technical Architecture

Adaptive Learning Web Platform for
Primary School Students with ADHD

Document	Backend Technical Architecture
Version	1.0
Date	14 October 2026
Repository	graduation-project (monorepo) · backend/
Baseline	main @ ac9a317 — 6/6 tests, typecheck and lint passing
Audience	Project team, supervisors, future contributors

Generated from the repository state at commit ac9a317 (main), 14 October 2026.

Contents

1. Executive Summary . . . 3
Status at a glance . . . 3
2. System Context . . . 4
Responsibilities split . . . 4
3. Technology Stack . . . 5
4. Architectural Style: Layered Architecture . . . 5
Layer responsibilities . . . 6
Dependency rules . . . 6
Transactions . . . 7
5. Request Lifecycle . . . 7
End-to-end example: health check . . . 8
Process lifecycle (server.ts) . . . 8
6. API Design . . . 8
URL structure and areas . . . 8
Response envelopes . . . 9
Error model . . . 9
Validation and pagination . . . 9
7. Cross-Cutting Concerns . . . 10
7.1 Configuration . . . 10
7.2 Logging and observability . . . 10
7.3 Security controls in place . . . 10
7.4 Child-data protection (requirement for the target system) . . . 10
8. Data Layer . . . 11
9. Target Domain Architecture (Proposed) . . . 12
9.1 Feature modules . . . 12
9.2 Proposed data model . . . 13
9.3 Adaptive engine . . . 13
9.4 Authentication and authorization . . . 14
9.5 Indicative endpoint catalogue . . . 15
10. Testing Strategy . . . 15
11. Build, Tooling and Deployment . . . 16
Recommended deployment topology . . . 16
12. Code Analysis Findings . . . 17
Strengths observed . . . 17
13. Implementation Roadmap . . . 18
Appendix A — Current folder structure . . . 18
Appendix B — Adding a feature (checklist) . . . 19

1. Executive Summary

This document describes the technical architecture of the backend for the **Adaptive Learning Web Platform for Primary School Students with ADHD**. It is based on a review of the repository documentation (root `README.md`, `backend/README.md`, `backend/docs/backend-folder-structure.md`) and of every source, configuration and test file in `backend/`.

The platform's guiding principle is **backend-driven logic**: the server is the single source of truth for scoring, rewards, progress and the adaptive engine. The React frontend only presents data and collects input, so a child (or anyone else) cannot alter learning results from the browser.

The backend today is a **production-grade skeleton**: a Node.js / Express 5 / TypeScript REST API with a strict layered architecture, validated configuration, structured logging, a uniform error model, rate limiting and a test harness. One feature, `GET /api/v1/health`, runs through every layer end to end. No domain models, authentication or learning features exist yet. Sections 1–8 document what is built; Section 9 proposes the target domain architecture; Sections 10–13 cover quality, operations, findings and the roadmap.

Status at a glance

Capability	Status	Notes
Layered structure (routes → controllers → services → repositories)	DONE	Demonstrated by the health feature
Config validation (Zod), CORS, helmet, rate limiting	DONE	Server refuses to start on invalid env
Structured logging + request correlation id	DONE	pino / pino-http, X-Request-Id
Uniform success / error envelopes, typed errors	DONE	Single error handler
Request validation middleware	DONE	validate({ body, query, params })
Test harness (Vitest + supertest)	DONE	6 tests, 2 files, all passing
Prisma 7 + PostgreSQL wiring	SCAFFOLD	Client + adapter ready, schema has no models
Area routers (account / admin / webhooks)	SCAFFOLD	Mounted but empty, no auth guard
Authentication & role-based authorization	PLANNED	Provider not chosen (TODO in routes/index.ts)
Domain: survey, quizzes, adaptive engine, rewards, dashboards	PLANNED	See Section 9
API documentation (OpenAPI), CI pipeline, deployment	PLANNED	See Sections 11 and 13

2. System Context

The repository is a monorepo with two independently built applications. Three user roles (student, parent, teacher) use one React single-page application. The SPA talks to the backend only through the versioned REST API under `/api/v1`; the backend is the only component with database access.

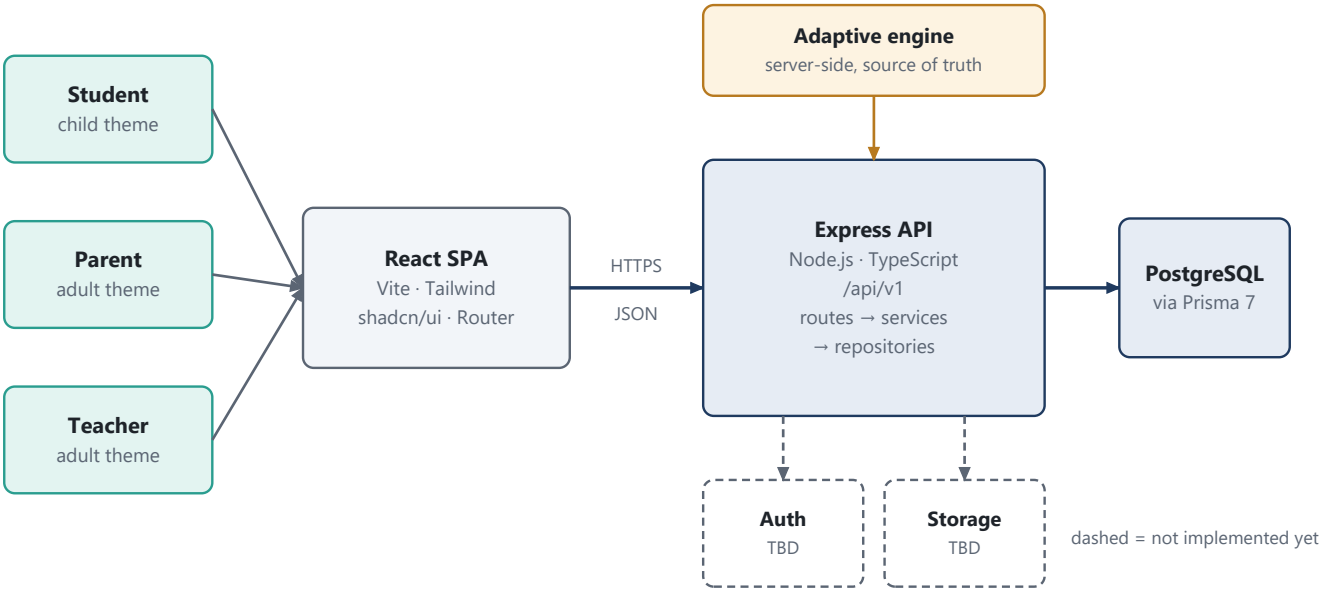


Figure 1 — System context. Dashed elements are planned but not implemented.

Responsibilities split

Concern	Frontend (React SPA)	Backend (Express API)
Rendering & themes	Child / adult themes, animations (Framer Motion, GSAP)	—
Input validation	UX-level (react-hook-form + Zod 3)	**Authoritative** (Zod 4 in validate())
Scoring, rewards, progress	Displays results only	**Computes and stores** all results
Adaptive decisions	Requests the next activity	**Chooses** difficulty and next activity
Access control	Hides navigation per role	**Enforces** role and ownership on every request
Persistence	None (session state only)	PostgreSQL via Prisma

3. Technology Stack

Concern	Choice	Version	Why
Runtime	Node.js	≥ 20.19	Required by Prisma 7; LTS
HTTP framework	Express	5.2	Native async error propagation, mature ecosystem
Language	TypeScript (strict)	6.0.x (pinned)	typescript-eslint does not support TS 7 yet
ORM	Prisma + @prisma/adapter-pg	7.10	Type-safe queries, migrations; driver-adapter mode
Database	PostgreSQL	—	Relational integrity for users, attempts, ledgers
Validation	Zod	4.6	Env validation and request schemas with inferred types
Security	helmet · cors · express-rate-limit	8.3 · 2.8 · 8.7	Headers, origin allow-list, abuse limits
Logging	pino · pino-http (pino-pretty in dev)	10.4 · 11.0	Fast structured JSON logs
Testing	Vitest · supertest	5.0 · 7.3	Unit + HTTP tests without opening a port
Tooling	ESLint 10, Prettier 3, tsx, tsc-alias	—	Lint rules enforce architecture (e.g. process.env ban)

Module system. The project compiles to CommonJS (`module: nodenext` without `"type": "module"`) and the Prisma generator is set to `moduleFormat = "cjs"`. The `@/` alias is resolved by tsx in development, by Vitest's alias in tests, and rewritten by `tsc-alias` in the production build.

4. Architectural Style: Layered Architecture

Every feature is built from the same layers, and a request only ever travels **downward**. Each layer has one reason to change: URLs, HTTP shape, business rules, or queries. Cross-cutting modules support the layers but contain no feature logic.

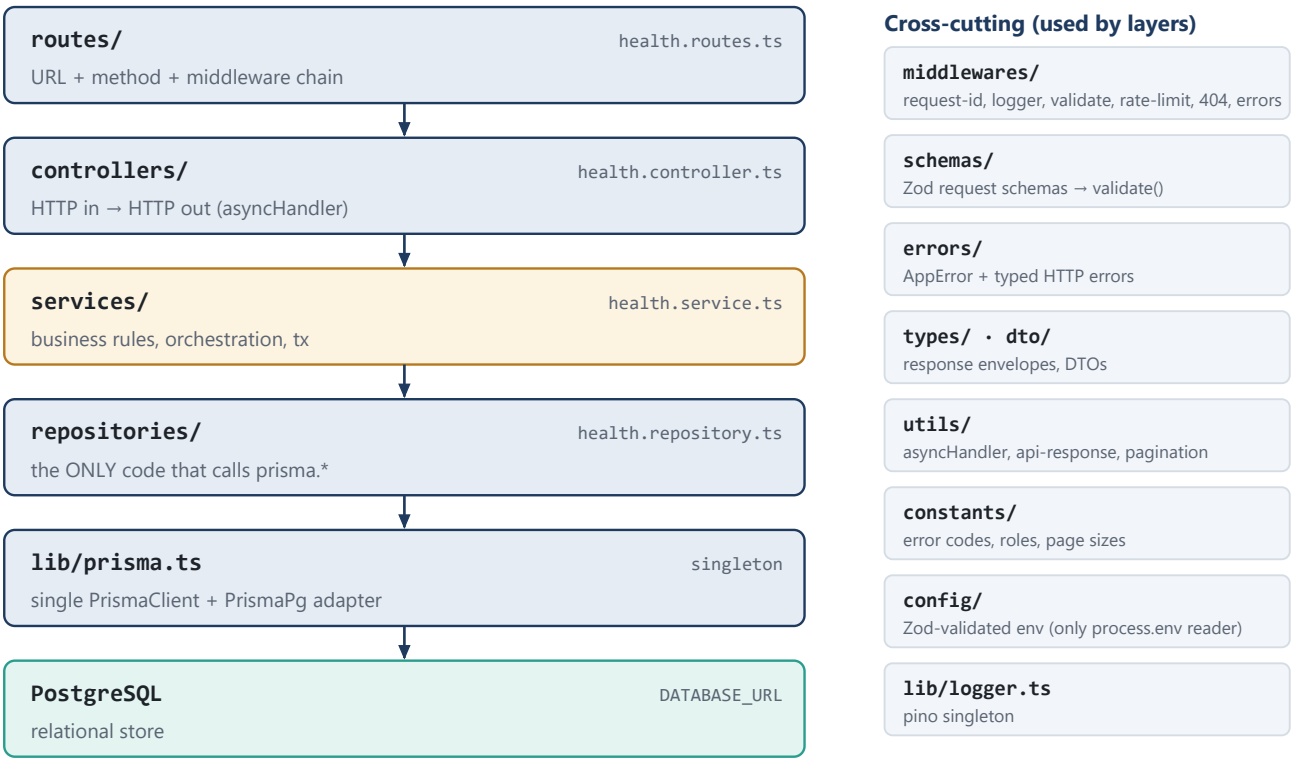


Figure 2 — Layers (left) and cross-cutting modules (right), with the health feature as the reference implementation.

Layer responsibilities

Layer	Does	Must not
routes/	Declare path, method, middleware (validate, limits), controller	Contain logic, Prisma calls or try/catch
controllers/	Read req, call **one** service method, choose status, send envelope	Hold business rules, import repositories/Prisma, catch errors
services/	Business rules, invariants, orchestration, transactions, DTO mapping	Import express or call prisma.* directly
repositories/	One Prisma query (or tightly related set), select/include shapes	Throw domain errors, call other repositories or services
schemas/	Zod schemas + inferred input types for each entity	Import anything except zod and constants

Dependency rules

A layer may import only from the layer directly below it. Services may call other services as long as the graph stays acyclic. These rules are partly enforced by ESLint (the `no-restricted-properties` rule blocks `process.env` outside `config/env.ts`; `consistent-type-imports` keeps type-only imports erasable).

```
routes  ──  controllers  ──  services  ──  repositories  ──  lib/prisma  ──  PostgreSQL
                                 │
                                 └──  errors, constants, lib (non-Prisma), other services
```

Transactions

Repositories never start a transaction. Each repository method takes an optional trailing `db: Db` argument (`Prisma.TransactionClient | typeof prisma`, defined in `base.repository.ts`). A service opens the transaction and passes the client down:

```
// services/quiz-attempt.service.ts  (illustrative)
await prisma.$transaction(async (tx) => {
  const attempt = await attemptRepository.markSubmitted(attemptId, score, tx)
  await rewardRepository.addPoints(attempt.studentId, points, "QUIZ", attempt.id, tx)
  await masteryRepository.upsert(attempt.studentId, skillId, nextMastery, tx)
})
```

5. Request Lifecycle

`src/app.ts` builds the Express application without calling `listen()`, so tests import it directly. `src/server.ts` starts the HTTP server and owns process lifecycle. The middleware order is deliberate:

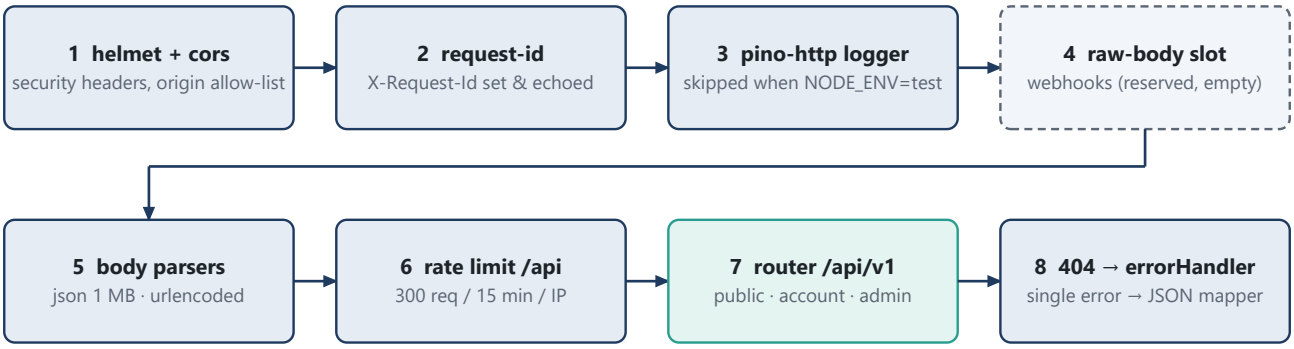


Figure 3 — Middleware pipeline as registered in src/app.ts.

Stage	Implementation detail
App settings	`x-powered-by` disabled; `trust proxy = 1` so the rate limiter sees the real client IP behind one proxy hop.
helmet / cors	Default helmet headers. CORS origins come from `CORS_ORIGINS` (comma-separated), `credentials: true` to allow cookie-based auth later.
request-id	Reuses an incoming `X-Request-Id` if ≤ 128 chars, otherwise generates a UUID; echoed in the response and used as the pino-http request id.
Raw-body slot	Reserved position for routes that need the unparsed body (signed webhooks) — must stay before `express.json()`.
Body parsing	`express.json({ limit: "1mb" })` and urlencoded. Malformed JSON is mapped to 400, not 500.
Rate limiting	`globalRateLimit`: 300 requests / 15 min / IP on `/api`; `strictRateLimit` (20 / 15 min) is exported for login-type routes. IETF draft-8 headers.
Error handling	`notFound` raises `NotFoundError`; `errorHandler` is the only place errors become responses.

End-to-end example: health check

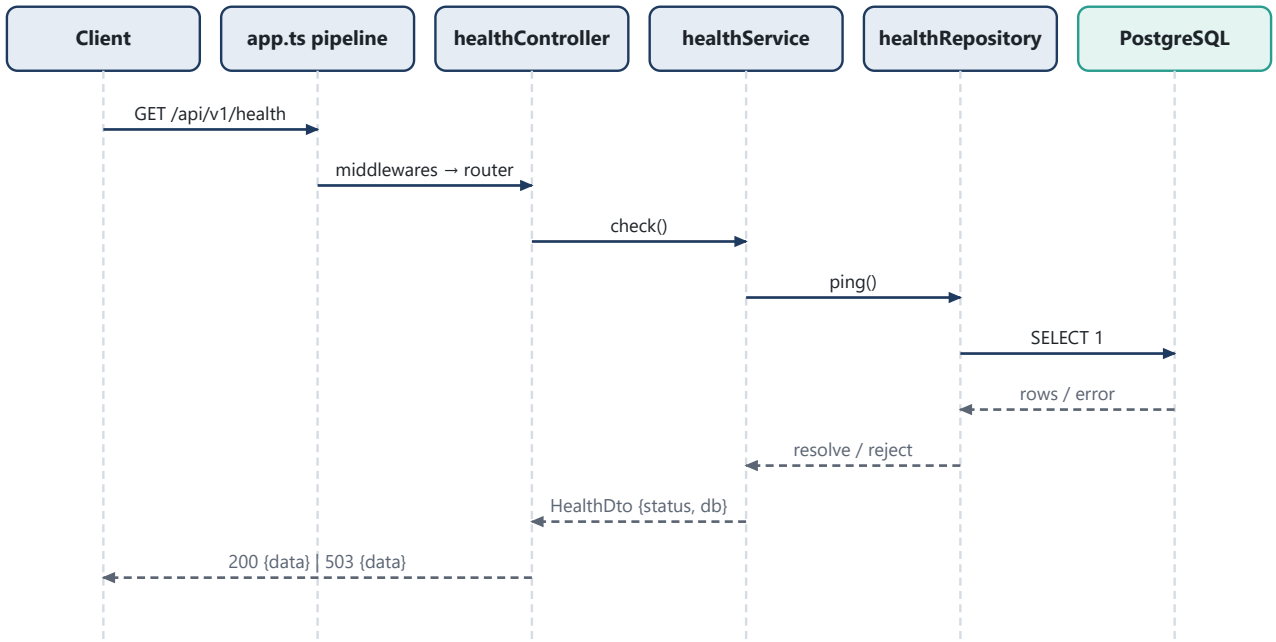


Figure 4 — GET /api/v1/health travelling through every layer.

`healthService.check()` converts a repository failure into `database: "down"` instead of throwing, so the endpoint answers **503** with a body when PostgreSQL is unreachable and **200** otherwise. This makes it suitable as a load-balancer readiness probe.

Process lifecycle (server.ts)

- **Graceful shutdown** on SIGINT / SIGTERM: stop accepting connections, wait for in-flight requests, `prisma.$disconnect()`, then exit. A 10-second unref'd timer forces exit if connections do not drain.
- **uncaughtException** is logged as fatal and the process exits with code 1 (a supervisor should restart it).
- **unhandledRejection** is logged but the process keeps running (see finding F-09).

6. API Design

URL structure and areas

All endpoints live under `/api/v1/<area>/<resource>`. Authentication and role checks are meant to be attached **once** where an area is mounted in `routes/index.ts`, not repeated per route.

Area (mount)	Audience	Current contents	Guard (target)
/ (public)	Anyone	`GET /health`	none
/account	Any signed-in user	empty router	authenticate
/admin	Privileged users	empty router	authenticate + requireRole(...)
/webhooks	External systems	folder only, not mounted	signature verification

> The current areas were inherited from a template whose roles were ADMIN / CUSTOMER. This platform's roles are STUDENT, PARENT and TEACHER, so Section 9.4 recommends role-named areas (`/student`, `/parent`, `/teacher`) instead of `/admin`.

Response envelopes

Every response uses one of two shapes, produced by the helpers in `utils/api-response.ts` (ok, `created`, `noContent`, `paginated`) and by the error handler:

```
// success                                    // error
{                                             {
  "data": { ... } | [ ... ],                    "error": {
  "meta": { "page": 1,                            "code": "VALIDATION_ERROR",
            "pageSize": 20,                       "message": "Request validation failed",
            "total": 134 }   // lists only         "details": [ { "location": "body",
}                                                              "path": "answers.0",
                                                             "message": "Required" } ]
                                                  }
                                              }
```

Error model

Services throw subclasses of `AppError` (status code, machine-readable code, message, optional details). Anything else is logged with the request id and returned as a generic 500 with no internals.

Class	HTTP	code	Typical use
BadRequestError	400	BAD_REQUEST	Malformed request, malformed JSON body
ValidationError	400	VALIDATION_ERROR	Zod failure in validate(); one entry per issue
UnauthorizedError	401	UNAUTHORIZED	Missing / invalid session
ForbiddenError	403	FORBIDDEN	Wrong role, or not the owner (e.g. parent of another child)
NotFoundError	404	NOT_FOUND	Unknown route or resource
ConflictError	409	CONFLICT	Duplicate (e.g. email, already-submitted attempt)
UnprocessableError	422	UNPROCESSABLE	Valid shape but violates a business rule
TooManyRequestsError	429	TOO_MANY_REQUESTS	Rate limiter
(any other error)	500	INTERNAL_ERROR	Logged; generic message returned

Validation and pagination

- `validate({ params, query, body })` parses each part with Zod and **replaces** it with the parsed, coerced and defaulted value. Because Express 5 exposes `req.query` as a getter, the middleware redefines the property.
- `common.schema.ts` provides `idParamSchema`, `paginationSchema` (page ≥ 1, pageSize 1–100, default 20), `sortOrderSchema` and `slugSchema`.
- `utils/pagination.ts` maps page/pageSize to Prisma `skip/take` and builds the meta object.
- Responses are **DTOs, not Prisma rows** (`types/dto/`), so answer keys, password hashes and internal flags can never leak by accident.

7. Cross-Cutting Concerns

7.1 Configuration

`config/env.ts` is the **only** reader of `process.env` (enforced by ESLint). It loads `.env` via dotenv, validates with Zod, prints each invalid variable and exits with code 1. The rest of the code imports a typed, frozen `config` object.

Variable	Default	Validation	Purpose
NODE_ENV	development	development | test | production	Mode; disables request logging in test
PORT	5000	positive integer (coerced)	HTTP port
LOG_LEVEL	info	fatal … trace | silent	pino level
CORS_ORIGINS	http://localhost:5173	string, comma-separated	Allowed browser origins (Vite dev server)
DATABASE_URL	— (required)	URL	PostgreSQL connection string

7.2 Logging and observability

- A single pino logger (`lib/logger.ts`), JSON in production, pretty-printed in development.
- pino-http logs every request with the correlation id; level is *error* for 5xx, *warn* for 4xx, *info* otherwise.
- Clients may send `X-Request-Id`; the same id appears in the response header and all log lines, which lets the frontend attach it to bug reports.
- **Recommendation:** configure pino `redact` for `req.headers.authorization`, cookies and any child-identifying fields before authentication is added.

7.3 Security controls in place

Control	Where	Effect
Security headers	helmet()	HSTS, noSniff, frameguard, CSP defaults, etc.
Origin allow-list	config/cors.ts	Only configured origins may call the API with credentials
Fingerprint reduction	app.disable('x-powered-by')	Hides the framework
Payload limit	express.json 1 MB	Mitigates large-body abuse
Rate limiting	middlewares/rate-limit.ts	Global 300/15 min; strict 20/15 min for sensitive routes
Input validation	validate() + Zod	Rejects unexpected shapes before controllers run
Error hygiene	error-handler.ts	No stack traces or internals in responses
Secrets	.env (gitignored), .env.example	Secrets never committed

7.4 Child-data protection (requirement for the target system)

Users are primary-school children and the data includes attention-related information, which is sensitive. The backend should apply: data minimisation (no real names required for students; birth year instead of date), verifiable parental consent recorded on the parent–student link, role- *and* ownership-based access on every read, encrypted transport (TLS) and storage-level encryption, retention and deletion endpoints for parents, and audit logging of teacher/parent access to a child's records.

8. Data Layer

Item	Current implementation
Schema	`prisma/schema.prisma` — generator + datasource only, **no models yet**
Generator	`prisma-client`, output `src/generated/prisma` (gitignored, regenerated on `npm install`), CJS, `.js` import extensions
Config	`prisma.config.ts` (Prisma 7) holds schema path, migrations path, seed command and `DATABASE_URL`
Client	`lib/prisma.ts` — one `PrismaClient` using the `PrismaPg` driver adapter; import from `@/generated/prisma/client`
Access rule	Only repositories import `lib/prisma` (seed script and server shutdown excepted)
Migrations	`npm run prisma:migrate` (migrate dev); folder currently empty
Seed	`prisma/seed.ts` via `npm run db:seed`; placeholder

Conventions to adopt with the first models: `cuid()` or UUID primary keys, `createdAt/updatedAt` on every table, `@@index` on every foreign key used for filtering, enums for roles and statuses, and a **ledger** table (append-only) for reward points instead of a mutable balance column.

9. Target Domain Architecture (Proposed)

> This section is a **proposal** derived from the root README (three roles; survey, quizzes, rewards and dashboards features; server-side adaptive engine and scoring). Nothing in it exists in code yet. It is designed to slot into the existing layers without structural change.

9.1 Feature modules

Module	Responsibility	Main service(s)	Primary area
auth	Sign-in, tokens, sessions, student login codes	auth.service	public
users / profiles	Accounts, student profile, parent–child links, consent	user.service, student-profile.service	parent, teacher
classrooms	Classes, join codes, enrolments	classroom.service	teacher
survey	Baseline & check-in questionnaires → learner profile	survey.service	student, parent
content	Skills, quizzes, questions (answer keys server-only)	quiz.service, skill.service	teacher
attempts	Start / answer / submit, server scoring, idempotency	quiz-attempt.service	student
adaptive engine	Mastery model, next activity & difficulty, break prompts	adaptive.service	student
rewards	Points ledger, badges, streaks	reward.service	student
dashboards	Read-only aggregates for parents and teachers	dashboard.service	parent, teacher

9.2 Proposed data model

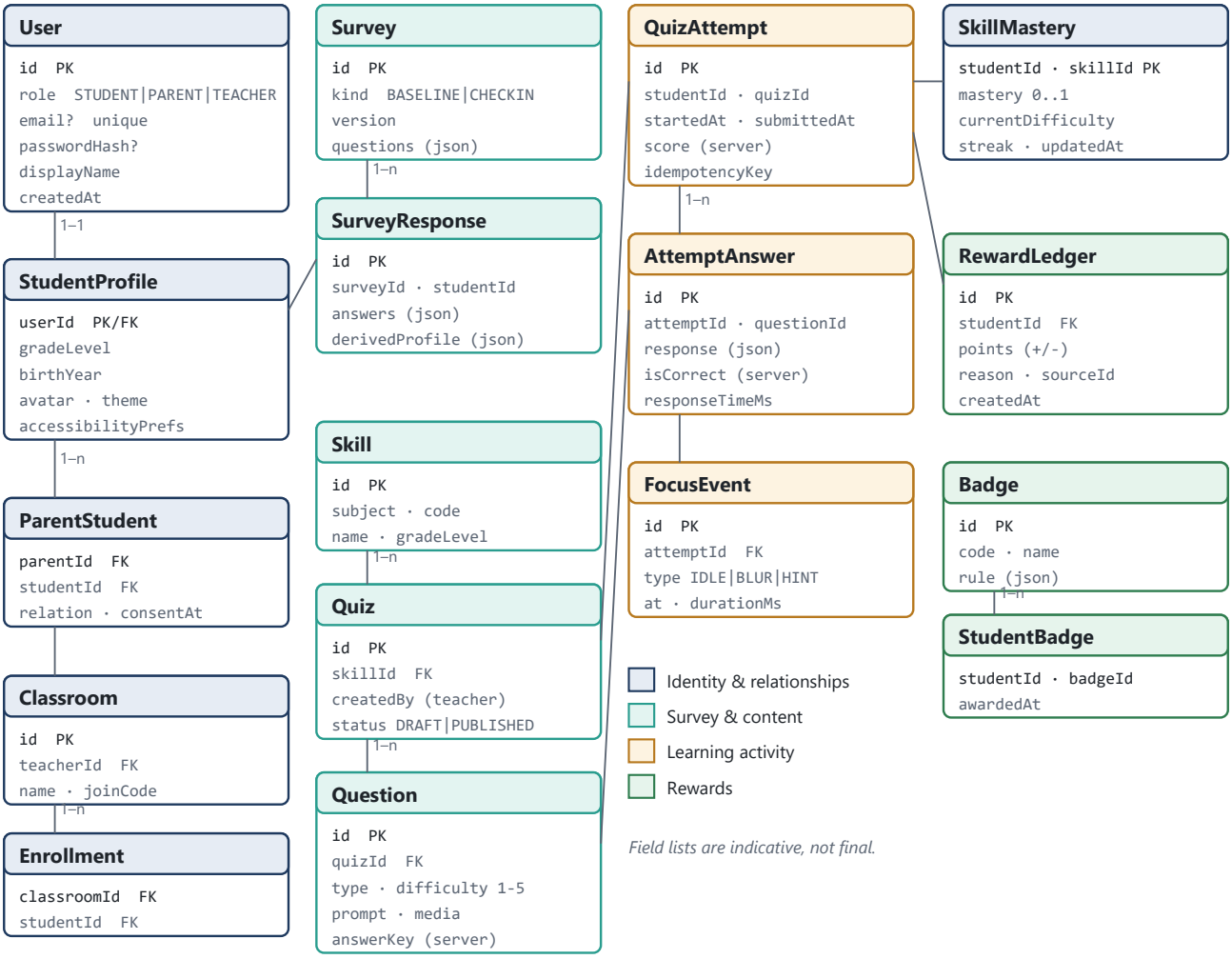


Figure 5 — Proposed entities and relationships (indicative).

9.3 Adaptive engine

The engine lives entirely in the service layer (`adaptive.service.ts`) as pure, unit-testable functions, with repositories providing state. A simple, explainable rule-based model is recommended first (e.g. raise difficulty after 3 correct answers within the expected time, lower it after 2 errors, suggest a short break after repeated idle/blur events), with thresholds kept in `constants/adaptive.ts` so they can later be replaced by a statistical model (e.g. Elo / Bayesian Knowledge Tracing) without changing the API.

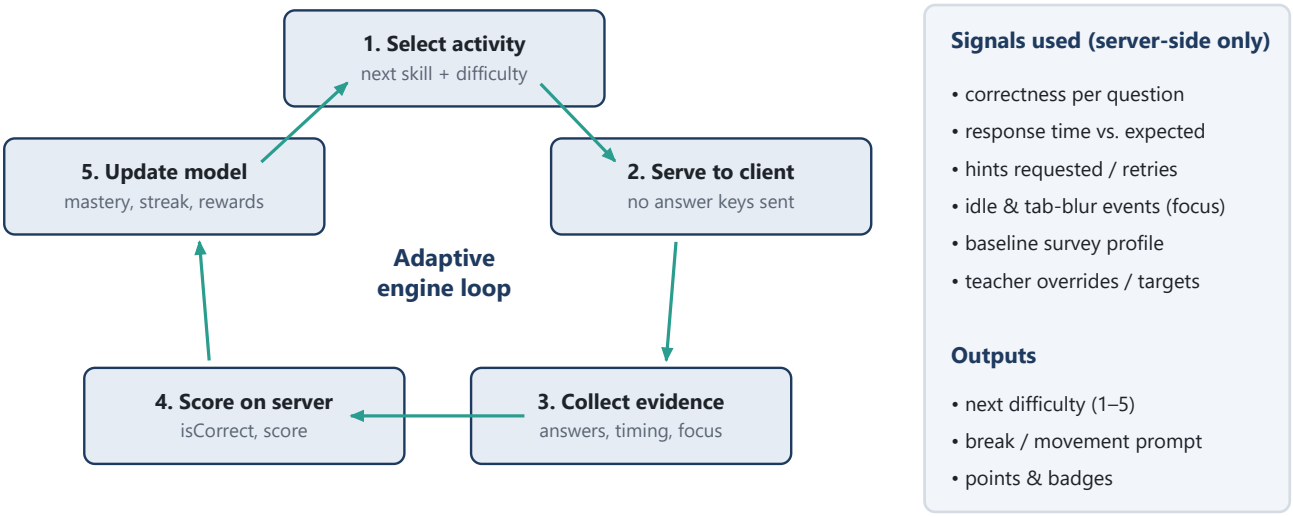


Figure 6 — Adaptive engine loop. All scoring and decisions happen on the server.

Integrity rules

- The client sends **raw answers and timings only**; it never sends a score, points or difficulty.
- Answer keys are never included in question DTOs.
- Attempt submission is **idempotent** (idempotency key / unique constraint); a second submit returns 409 or the stored result.
- Score, mastery update and reward ledger entries are written in **one transaction**.
- Response times are measured on the server between "question served" and "answer received"; client timings are advisory.

9.4 Authentication and authorization

No provider has been chosen yet (`TODO` in `routes/index.ts`). Because young children cannot be expected to manage email and passwords, a self-hosted approach fits well:

- **Parents / teachers:** email + password (argon2id hash), `strictRateLimit` on login.
- **Students:** teacher-issued class code + personal picture/PIN code, or a parent-initiated session on a shared device.
- **Tokens:** short-lived access JWT (≈15 min) plus a rotating refresh token in an `httpOnly, Secure, SameSite` cookie (CORS `credentials: true` is already set).
- **Middlewares:** `authenticate` sets `req.auth = { userId, role }` (declared in a new `types/express.d.ts`); `requireRole(...roles)` guards each area.
- **Ownership:** enforced in services (a parent may read only linked children; a teacher only students enrolled in their classes) and raised as `ForbiddenError`.

```
// routes/index.ts  (target)
router.use("/",        publicRouter)                                  // /health, /auth/*
router.use("/student", authenticate, requireRole("STUDENT"), studentRouter)
router.use("/parent",  authenticate, requireRole("PARENT"),  parentRouter)
router.use("/teacher", authenticate, requireRole("TEACHER"), teacherRouter)
```

9.5 Indicative endpoint catalogue

Method & path	Role	Purpose
POST /auth/login · /auth/refresh · /auth/logout	public	Session management
POST /auth/student-login	public	Class code + PIN login for children
GET /student/me	STUDENT	Profile, theme, points balance
GET /student/surveys/pending · POST /student/surveys/:id/responses	STUDENT	Baseline / check-in survey
GET /student/activities/next	STUDENT	Adaptive engine picks the next quiz + difficulty
POST /student/quizzes/:id/attempts	STUDENT	Start attempt (returns questions without keys)
POST /student/attempts/:id/answers · /submit	STUDENT	Record answers; submit → server scoring
GET /student/rewards	STUDENT	Points history and badges
GET /parent/children · /parent/children/:id/progress	PARENT	Linked children and progress dashboard
GET · POST · PATCH /teacher/classes(/:id/students)	TEACHER	Class and enrolment management
CRUD /teacher/quizzes · /teacher/quizzes/:id/questions	TEACHER	Content authoring
GET /teacher/classes/:id/dashboard	TEACHER	Class-level aggregates and alerts

10. Testing Strategy

Vitest runs every `tests/**/*.test.ts` with `NODE_ENV=test` and `LOG_LEVEL=silent`. supertest calls the exported app in-process.

Level	Folder	Approach	Current tests
Unit — utils	tests/unit/utils	Pure functions	slugify (2)
Unit — services	tests/unit/services	Repositories mocked with vi.spyOn / fakes	—
Integration — repositories	tests/integration/repositories	Real test database, reset per suite	—
Integration — routes	tests/integration/routes	supertest against app, repositories mocked	health 200 / 503, 404 envelope, malformed JSON (4)

Verification run during this review: **2 test files, 6 tests passed**; `tsc --noEmit` and `eslint .` reported no errors.

Priorities as features land

- Exhaustive unit tests for the adaptive engine and scoring (table-driven: inputs → expected difficulty/score).
- Authorization tests for every protected route: wrong role → 403, other family's child → 403, no token → 401.
- Repository tests against a disposable PostgreSQL (Docker / Testcontainers) for transactions and constraints.
- Coverage threshold in CI (e.g. 80 % on `services/`).

11. Build, Tooling and Deployment

Script	Command	Purpose
dev	tsx watch src/server.ts	Hot-reload development server
build	tsc -p tsconfig.build.json && tsc-alias	Compile src → dist, rewrite @/ aliases
start	node dist/server.js	Run the compiled server
typecheck · lint · format	tsc --noEmit · eslint . · prettier	Static quality gates
test · test:watch	vitest run · vitest	Test suite
prisma:generate · migrate · studio	prisma …	Client generation, migrations, DB browser
db:seed	prisma db seed	Seed data (tsx prisma/seed.ts)
postinstall	prisma generate	Generated client always present after install

Recommended deployment topology

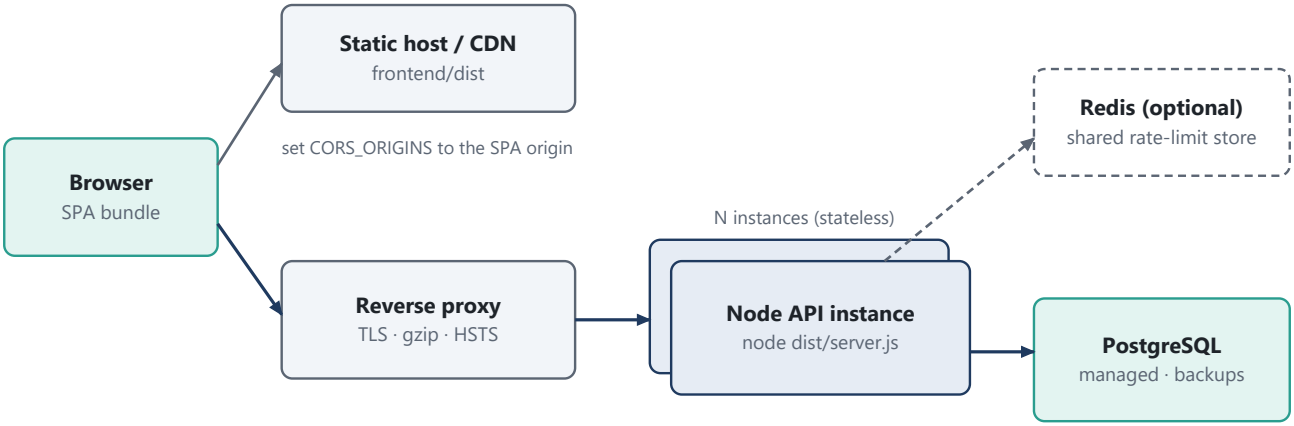


Figure 7 — Target deployment. The API is stateless, so it scales horizontally.

Recommended CI pipeline

```
npm ci  →  npm run format:check  →  npm run lint  →  npm run typecheck
        →  npm test  (DATABASE_URL provided as a CI variable)
        →  npm run build  →  prisma migrate deploy  (on release only)
```

12. Code Analysis Findings

Findings from reviewing the backend source, configuration and documentation. None are defects in the code that exists today; they are gaps and risks to address before or while building features.

#	Finding	Severity	Recommendation
F-01	No authentication. `/account` and `/admin` are mounted without any guard (TODO in `routes/index.ts`). Harmless while empty, unsafe as soon as a route is added.	HIGH	Implement authenticate / requireRole before the first non-public route (Section 9.4).
F-02	`docs/backend-folder-structure.md` describes a different product (ECU store: Clerk, Stripe, R2, brands, firmware, ADMIN/CUSTOMER).	HIGH	Rewrite for this domain; keep the layer rules, replace examples with survey/quiz/attempt.
F-03	Prisma schema has no models and no migrations; no domain logic exists yet.	MEDIUM	Model identity + relationships first (Section 9.2), then content and attempts.
F-04	Area naming (`admin`) does not match the roles STUDENT / PARENT / TEACHER; there is no ADMIN role.	MEDIUM	Use role-named routers (/student, /parent, /teacher) or add an explicit ADMIN role.
F-05	Tests depend on a local `.env`: `env.ts` exits if `DATABASE_URL` is missing, and vitest only sets NODE_ENV and LOG_LEVEL.	MEDIUM	Add a dummy DATABASE_URL to `vitest.config.mts` env, or provide it in CI.
F-06	Rate limiter uses the default in-memory store: limits are per process and reset on restart.	MEDIUM	Use a shared store (Redis / PostgreSQL) once more than one instance runs.
F-07	No `types/express.d.ts` yet for `req.auth`.	LOW	Add with the auth middleware.
F-08	No API contract documentation (OpenAPI).	LOW	Generate OpenAPI from the Zod schemas (e.g. zod-openapi) and serve Swagger UI in development.
F-09	`unhandledRejection` only logs; the process may continue in an inconsistent state.	LOW	Treat like uncaughtException (log fatal, graceful shutdown) once a process manager is in place.
F-10	Frontend uses Zod 3, backend Zod 4; schemas cannot be shared as-is.	LOW	Align versions or add a small shared package for request schemas.
F-11	No log redaction configured.	LOW	Add pino `redact` for auth headers, cookies and child-identifying fields.
F-12	Root README frontend snippet uses a broken code fence (`` `ash ``).	LOW	Replace with a proper ```` ```bash ```` block.

Strengths observed

- Clear, documented layering with a working reference feature that can be copied.
- Fail-fast configuration and a single, lint-enforced entry point for environment variables.
- One error path: typed errors, a uniform envelope, no internal leakage, malformed JSON handled.
- `app.ts` / `server.ts` split enables fast in-process HTTP tests.
- Production concerns already present: graceful shutdown, request ids, rate limiting, proxy awareness, readiness-style health check.
- Strict TypeScript (`noUncheckedIndexedAccess`, `noImplicitOverride`) with clean typecheck and lint.

13. Implementation Roadmap

Phase	Scope	Exit criteria
0 · Foundation fixes	F-02 doc rewrite, F-05 test env, CI pipeline, OpenAPI skeleton, log redaction	CI green on every PR
1 · Identity & access	User, StudentProfile, ParentStudent, Classroom, Enrollment; auth service, authenticate / requireRole, role-named routers	401/403 tests pass for every area
2 · Survey	Baseline survey, responses, derived learner profile	Student completes survey; profile stored
3 · Content & attempts	Skills, quizzes, questions (teacher CRUD); attempt start / answer / submit with server scoring	Idempotent submit, keys never exposed
4 · Adaptive engine	SkillMastery, next-activity endpoint, rule-based difficulty and break prompts	Table-driven engine tests pass
5 · Rewards	Points ledger, badges, streaks — written in the attempt transaction	Ledger balances reconcile
6 · Dashboards	Parent and teacher read-only aggregates	Ownership rules verified
7 · Hardening & release	Shared rate-limit store, backups, retention/deletion, load test, deployment	Production checklist signed off

Appendix A — Current folder structure

```
backend/
├── docs/backend-folder-structure.md   # layer rules (domain examples outdated, see F-02)
├── prisma/
│   ├── schema.prisma                  # generator + datasource, no models
│   ├── migrations/                    # empty
│   └── seed.ts
├── prisma.config.ts                   # Prisma 7 config (schema, migrations, seed, DATABASE_URL)
├── src/
│   ├── server.ts                      # listen + graceful shutdown + process handlers
│   ├── app.ts                         # createApp(): middleware pipeline, no listen
│   ├── config/        env.ts · cors.ts · index.ts
│   ├── routes/        index.ts · public/{index, health.routes} · account/ · admin/ · webhooks/
│   ├── controllers/   public/health.controller.ts
│   ├── services/      health.service.ts
│   ├── repositories/  base.repository.ts (Db type) · health.repository.ts
│   ├── schemas/       common.schema.ts
│   ├── middlewares/   request-id · request-logger · validate · rate-limit · not-found · error-handler
│   ├── lib/           prisma.ts · logger.ts
│   ├── errors/        app-error.ts · http-errors.ts · index.ts
│   ├── types/         api.ts · dto/health.dto.ts
│   ├── utils/         async-handler · api-response · pagination · slugify
│   ├── constants/     error-codes · pagination · roles
│   └── generated/prisma/              # generated client (gitignored)
└── tests/
    ├── unit/{services,utils}/          # slugify.test.ts
    ├── integration/{repositories,routes}/  # app.routes.test.ts
    └── fixtures/
```

Appendix B — Adding a feature (checklist)

Step	File	Notes
1	prisma/schema.prisma → npm run prisma:migrate	Model, relations, indexes
2	src/repositories/<feature>.repository.ts	Only place prisma.* is used; trailing db param
3	src/schemas/<feature>.schema.ts	Zod schemas + z.infer input types
4	src/types/dto/<feature>.dto.ts	Response shapes; never return raw rows
5	src/services/<feature>.service.ts	Rules, ownership checks, transactions; throws AppErrors
6	src/controllers/<area>/<feature>.controller.ts	asyncHandler; one service call; ok/created/paginated
7	src/routes/<area>/<feature>.routes.ts	validate({ params, query, body }); optional strictRateLimit
8	src/routes/<area>/index.ts	Mount the feature router
9	tests/…	Service unit tests → repository integration → route tests incl. 401/403